File System

There are 5 types of Windows file systems: FAT12, FAT16, FAT32, NTFS, and exFAT. FAT12 and FAT16 are no longer used on modern Windows operating systems. We will touch upon the FAT32 and exFAT file systems for this training, but our main focus will be the NTFS file system.

FAT32 (File Allocation Table) is widely used across many types of storage devices such as USB memory sticks and SD cards but can also be used to format hard drives. The "32" in the name refers to the fact that FAT32 uses 32 bits of data for identifying data clusters on a storage device.

`Pros of FAT32:`

- Device compatibility - it can be used on computers, digital cameras, gaming consoles, smartphones, tablets, and more.
- Operating system cross-compatibility - It works on all Windows operating systems starting from Windows 95 and is also supported by MacOS and Linux.

`Cons of FAT32:`

- Can only be used with files that are less than 4GB.

- No built-in data protection or file compression features.
- Must use third-party tools for file encryption.

NTFS (New Technology File System) is the default Windows file system since Windows NT 3.1. In addition to making up for the shortcomings of FAT32, NTFS also has better support for metadata and better performance due to improved data structuring.

`Pros of NTFS:`

- NTFS is reliable and can restore the consistency of the file system in the event of a system failure or power loss.
- Provides security by allowing us to set granular permissions on both files and folders.
- Supports very large-sized partitions.
- Has journaling built-in, meaning that file modifications (addition, modification, deletion) are logged.

`Cons of NTFS:`

- Most mobile devices do not support NTFS natively.
- Older media devices such as TVs and digital cameras do not offer support for NTFS storage devices.

Permissions

The NTFS file system has many basic and advanced permissions. Some of the key permission types are:

Permission Type	Description
Full Control	Allows reading, writing, changing, deleting of files/folders.
Modify	Allows reading, writing, and deleting of files/folders.
List Folder Contents	Allows for viewing and listing folders and subfolders as well as executing files. Folders only inherit this permission.
Read and Execute	Allows for viewing and listing files and subfolders as well as executing files. Files and folders inherit this permission.
Write	Allows for adding files to folders and subfolders and writing to a file.
Read	Allows for viewing and listing of folders and subfolders and viewing a file's contents.
Traverse Folder	This allows or denies the ability to move through folders to reach other files or folders. For example, a user may not have permission to list the directory contents or view files in the documents or web apps directory in this example c:\users\bsmith\documents\webapps\backups\backup_02042020.zip but with Traverse Folder permissions applied, they can access the backup archive.

Files and folders inherit the NTFS permissions of their parent folder for ease of administration, so administrators do not need to explicitly set permissions for each file and folder, as this would be extremely time-consuming. If permissions do need to be set explicitly, an administrator can

disable permissions inheritance for the necessary files and folders and then set the permissions directly on each.

Integrity Control Access Control List (icacls)

NTFS permissions on files and folders in Windows can be managed using the File Explorer GUI under the security tab. Apart from the GUI, we can also achieve a fine level of granularity over NTFS file permissions in Windows from the command line using the icacls utility.

We can list out the NTFS permissions on a specific directory by running either `icacls` from within the working directory or `icacls C:\Windows` against a directory not currently in.

```cmd-session
C:\htb> icacls c:\windows
c:\windows NT SERVICE\TrustedInstaller:(F)
           NT SERVICE\TrustedInstaller:(CI)(IO)(F)
           NT AUTHORITY\SYSTEM:(M)
           NT AUTHORITY\SYSTEM:(OI)(CI)(IO)(F)
           BUILTIN\Administrators:(M)
           BUILTIN\Administrators:(OI)(CI)(IO)(F)
           BUILTIN\Users:(RX)
           BUILTIN\Users:(OI)(CI)(IO)(GR,GE)
           CREATOR OWNER:(OI)(CI)(IO)(F)
           APPLICATION PACKAGE AUTHORITY\ALL APPLICATION PACKAGES:(RX)
           APPLICATION PACKAGE AUTHORITY\ALL APPLICATION PACKAGES:(OI)(CI)(IO)(GR,GE)
           APPLICATION PACKAGE AUTHORITY\ALL RESTRICTED APPLICATION PACKAGES:(RX)
           APPLICATION PACKAGE AUTHORITY\ALL RESTRICTED APPLICATION PACKAGES:(OI)(CI)
(IO)(GR,GE)
```

```
Successfully processed 1 files; Failed processing 0 files
```

The resource access level is listed after each user in the output. The possible inheritance settings are:

- `(CI)` : container inherit
- `(OI)` : object inherit
- `(IO)` : inherit only
- `(NP)` : do not propagate inherit
- `(I)` : permission inherited from parent container

In the above example, the `NT AUTHORITY\SYSTEM` account has object inherit, container inherit, inherit only, and full access permissions. This means that this account has full control over all file system objects in this directory and subdirectories.

Basic access permissions are as follows:

- `F` : full access
- `D` : delete access
- `N` : no access
- `M` : modify access
- `RX` : read and execute access
- `R` : read-only access
- `W` : write-only access

We can add and remove permissions via the command line using `icacls`. Here we are executing `icacls` in the context of a local administrator account showing the `C:\users` directory where the `joe` user does not have any write permissions.

```cmd-session
C:\htb> icacls c:\Users
c:\Users NT AUTHORITY\SYSTEM:(OI)(CI)(F)
         BUILTIN\Administrators:(OI)(CI)(F)
         BUILTIN\Users:(RX)
         BUILTIN\Users:(OI)(CI)(IO)(GR,GE)
         Everyone:(RX)
         Everyone:(OI)(CI)(IO)(GR,GE)

Successfully processed 1 files; Failed processing 0 files
```

Using the command `icacls c:\users /grant joe:f` we can grant the joe user full control over the directory, but given that `(oi)` and `(ci)` were not included in the command, the joe user will only have rights over the `c:\users` folder but not over the user subdirectories and files contained within them.

```cmd-session
C:\htb> icacls c:\users /grant joe:f
processed file: c:\users
Successfully processed 1 files; Failed processing 0 files
```

```cmd-session
```

```
C:\htb> >icacls c:\users
c:\users WS01\joe:(F)
         NT AUTHORITY\SYSTEM:(OI)(CI)(F)
         BUILTIN\Administrators:(OI)(CI)(F)
         BUILTIN\Users:(RX)
         BUILTIN\Users:(OI)(CI)(IO)(GR,GE)
         Everyone:(RX)
         Everyone:(OI)(CI)(IO)(GR,GE)

Successfully processed 1 files; Failed processing 0 files
```

These permissions can be revoked using the command `icacls c:\users /remove joe`.

`icacls` is very powerful and can be used in a domain setting to give certain users or groups specific permissions over a file or folder, explicitly deny access, enable or disable inheritance permissions, and change directory/file ownership.

A full listing of `icacls` command-line arguments and detailed permission settings can be found here.

Connect to HTB

Target(s)

Time left: 108 min(s)

10.129.201.57 (ACADEMY-MISC-WS01)

Enable step-by-step solutions PRO

Question 1 +20

What system user has full control over the c:\users directory?

RDP to **10.129.201.57 (ACADEMY-MISC-WS01),** with user "**htb-student**" and password "**Academy_WinFun!**"

Write your answer

Submit